

FOUNDATION AI-DRIVEN CLOUD & DEVSECOPS CAPSTONE

# Inventory Management System Cloud & DevSecOps Modernization

Customer: Acme Retail Ltd.

Ushasri D • Group 22 • Cloud & DevSecOps Engineer



# Customer Problem & Project Objectives

## CURRENT STATE — CHALLENGES

- ! Manual infrastructure provisioning with no repeatable process
- ! Manual, ad-hoc application deployments
- ! Missing Infrastructure as Code
- ! Inconsistent CI/CD across changes
- ! Weak security validation before release
- ! Poor documentation of architecture and decisions

## PROJECT OBJECTIVES

- ✓ Repeatable, modular Terraform-based infrastructure
- ✓ Automated, consistent deployment pipeline
- ✓ Full Infrastructure-as-Code coverage
- ✓ Standardized CI/CD for every change
- ✓ Layered, automated security scanning gates
- ✓ Clear specs, diagrams, and decision records

# Application Overview — Spring Boot IMS & E-Checkout



## Inventory

Stock levels and item catalog



## Members

Member accounts and profiles



## Purchase Orders

Supplier ordering workflow



## Receipts

Transaction receipts



## Discounts

Promotions and pricing rules



## Recommendations

Product recommendations



## Sales Analysis

Sales reporting and trends



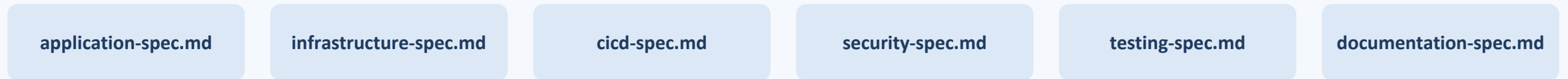
## E-Checkout

End-to-end checkout flow

# AI Engineering Specifications & Methodology



## Six AI Engineering Specifications



*Each specification guided AI-generated artifacts, which were then reviewed, secured, validated, and documented before acceptance.*

# AI-Assisted Engineering Approach

*GitHub Copilot and conversational AI assistants were used as engineering assistants — not as replacements for validation or decision-making.*

## WHERE AI ASSISTED

- 1 **Application development** — Spring Boot code, API structure, validation logic, and test generation
- 2 **Infrastructure as Code** — Drafted/refined Terraform for VPC, ECS Fargate, ALB, RDS, IAM, KMS, CloudWatch, autoscaling, S3 archive
- 3 **CI/CD** — Generated & troubleshoot GitHub Actions for Maven builds, security scanning, Docker builds, ECR publishing, ECS deployment
- 4 **Containers & orchestration** — Dockerfile, ECS task definitions, and Kubernetes/Kustomize manifests (local portability & security validation)
- 5 **Automation scripts** — Shell scripts for CloudWatch log archiving and one-off RDS backups to S3
- 6 **Security** — Interpreted Checkov, Trivy, and Gitleaks findings and proposed secure remediations
- 7 **Documentation** — Architecture diagrams, ADRs, operating guides, troubleshooting docs, presentation content

## HUMAN VALIDATION PROCESS



## AI-Assisted Engineering Approach — Verification

All AI-generated work was reviewed and verified using the same tooling applied to every change in the pipeline:

Maven tests

Terraform fmt & validate

Checkov

Trivy

Gitleaks

GitHub Actions CI

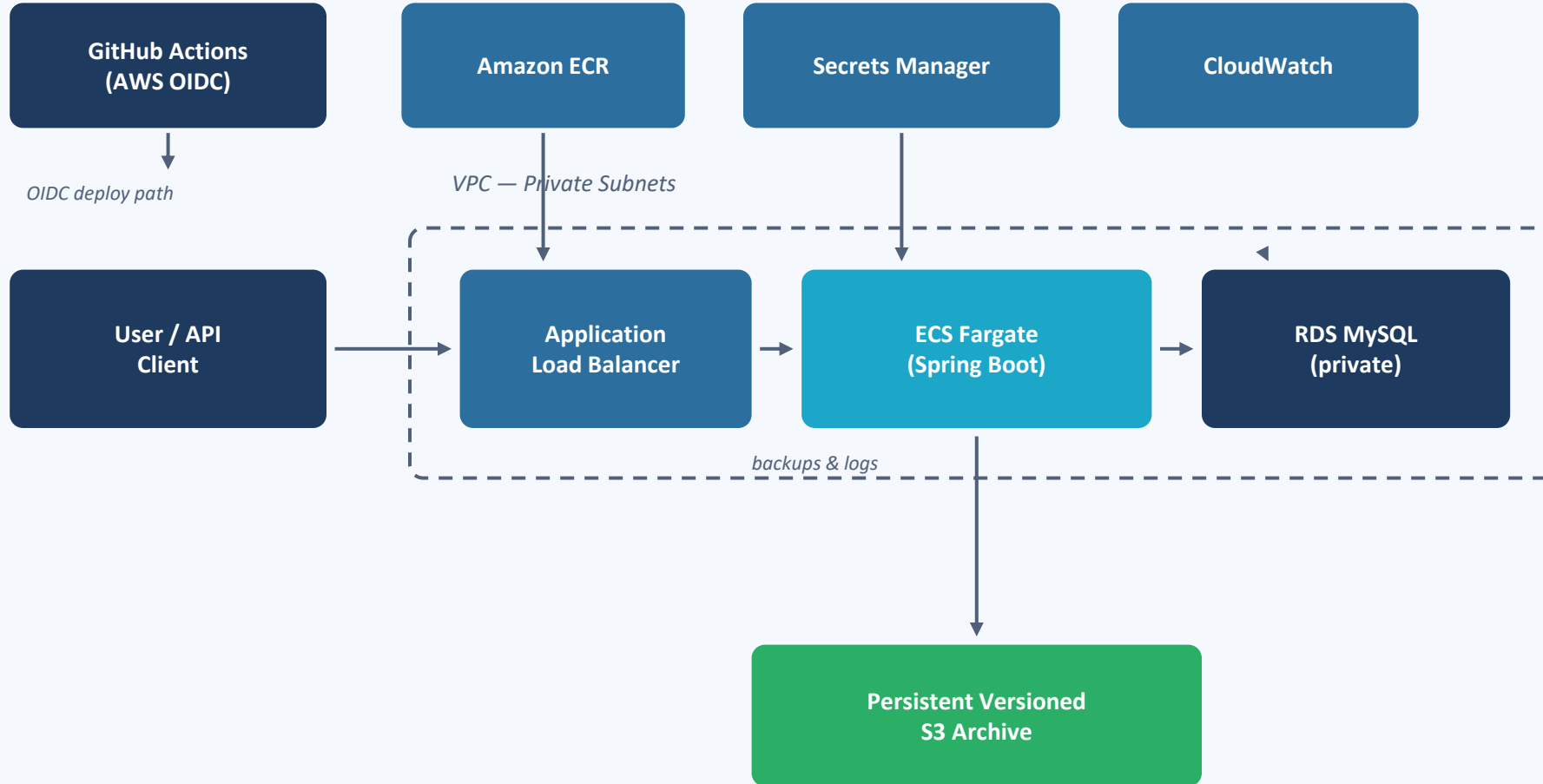
AWS runtime health checks

Load testing

Backup & restore testing

*“AI accelerated implementation and troubleshooting, while final architectural decisions, security controls, testing, and acceptance remained human-controlled.”*

# Final AWS Architecture



# Modular Terraform Across Three Environments

## Dev Environment

Recreated through approved workflow

Applied

## Production Environment

Terraform validated but not applied

Validated — Not Applied

## Persistent Archive

Independent state; survives environment teardown

Applied

Modular design: reusable Terraform modules for networking (VPC), compute (ECS Fargate), data (RDS), security (IAM, KMS, security groups), and observability (CloudWatch) are composed per environment.

# CI Pipeline — GitHub Actions Checks



**Security & quality gates run inline:** Gitleaks blocks committed secrets, Checkov policy-checks Terraform and rendered Kubernetes manifests, and Trivy blocks fixable high- and critical-severity image findings. SonarCloud Quality Gate passed with no new security issues or hotspots.

# CD Pipeline — Deployment to ECS Fargate



**No long-lived AWS keys are stored in GitHub.** Deployment only proceeds after CI succeeds on master and the ECS and ECR guard is set, authenticating via short-lived OIDC credentials before the new revision is deployed and monitored for stability.

# Security Design

1

## No Long-Lived Keys

GitHub OIDC issues short-lived AWS credentials

2

## IAM Least Privilege

Scoped roles and policies per component

3

## Private RDS

Database has no public endpoint

4

## Security-Group Isolation

Network access restricted by component

5

## Secrets Manager

Credentials never stored in code or CI

6

## Encryption

Data encrypted at rest and in transit

7

## Gitleaks / Checkov / Trivy

Secret, IaC, and image scanning in CI

8

## Documented Scan Exceptions

Any accepted findings are recorded, not hidden

**Future Work: Production HTTPS — ACM public certificates require a controlled domain, which has not yet been provisioned.**

# Autoscaling & Performance Evidence

ECS AUTOSCALING POLICY

**60%**  
CPU Target Utilization

**1**

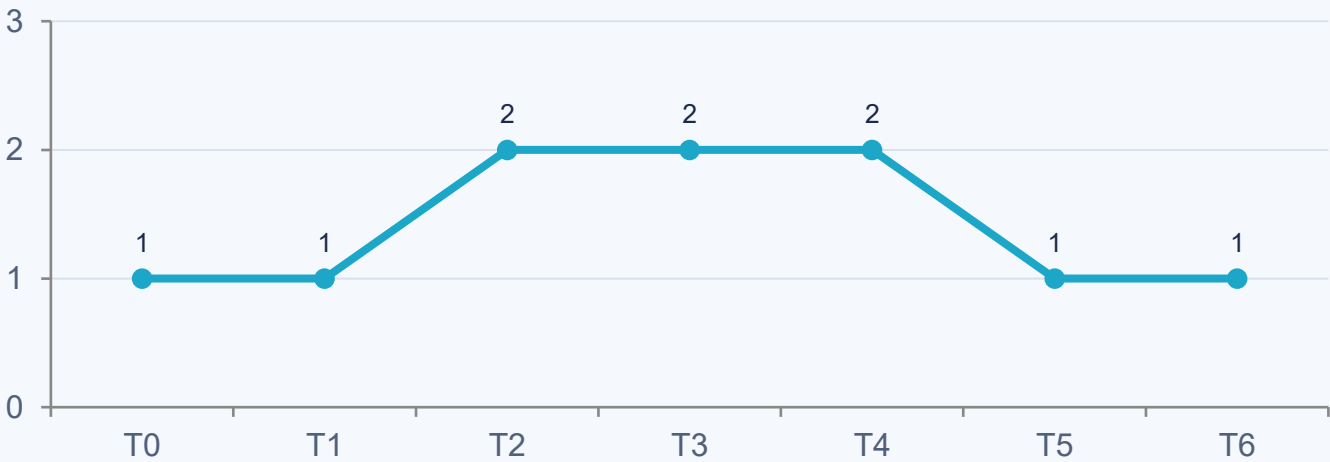
Minimum Tasks

**2**

Maximum Tasks

Scaling verified: 1 task → 2 tasks → back to 1 task under generated load, confirming both scale-out and scale-in behavior.

TASK COUNT DURING LOAD TEST



**7,507**

Load Test Requests

**0**

Failed Requests

**24.76**

Requests / sec

# Backup, Recovery & Log Archival

| Area                   | Evidence                                                                                            |
|------------------------|-----------------------------------------------------------------------------------------------------|
| Persistent S3 Archive  | Independent of environment lifecycle — public access blocked, AES256 encryption, versioning enabled |
| Log Retention          | Application and infrastructure logs retained for 30 days                                            |
| SQL Backup Retention   | Portable SQL backups retained for 90 days                                                           |
| Backup Mechanism       | One-off ECS Fargate task performs the database backup to S3                                         |
| Restore Validation     | Successful restore verified: 8 tables, 4 member records, 8 product records                          |
| Post-Teardown Evidence | Application and VPC flow logs preserved after dev infrastructure teardown                           |

*RDS automated backups remain primary. EventBridge Scheduler runs the portable Fargate SQL backup daily; S3 is the tested secondary recovery copy.*

# Engineering Decisions & Trade-offs

| Decision                                             | Rationale / Trade-off                                                   |
|------------------------------------------------------|-------------------------------------------------------------------------|
| ECS Fargate replaced the initial EKS proposal        | Reduced cost and operational complexity for a single service            |
| Kubernetes & Kustomize retained                      | Kept for local portability and CI policy scanning, not cloud deployment |
| S3 archive uses independent Terraform state          | Persists evidence independently of environment lifecycle                |
| Dev used HTTP temporarily                            | Production requires a domain, ACM certificate, and HTTPS                |
| Backup & log archival scripts are manually triggered | Scheduling automation is future work                                    |

# Results & Lessons Learned



Repeatable infrastructure via modular Terraform



Layered security validation across the pipeline



Tested backup and restore process



Disciplined review and refinement of AI-generated artifacts



Approved Terraform automation and CI/CD delivery



Verified application health and autoscaling



Persistent state, backups, and evidence retained across teardown

# Future Improvements & Conclusion

- 1 Production domain and HTTPS
- 2 Scheduled log archival and monitoring
- 3 Approved OWASP ZAP dynamic testing
- 4 Scheduled database backup and on-demand log archival
- 5 Production deployment and recovery drill
- 6 Centralized security monitoring

*A secure, automated, evidence-backed foundation — validated end to end and ready to extend toward production.*